

NodeJs e nossa primeira API

Sumário

- Revisão
- O que é o Node
- Setup inicial de um projeto
 - Type module
- instalação de pacotes
 - express
- Primeira rota

Relembrando o que já
aprendemos

API

Servidor

Rota

Status HTTP

Faixas

Método HTTP

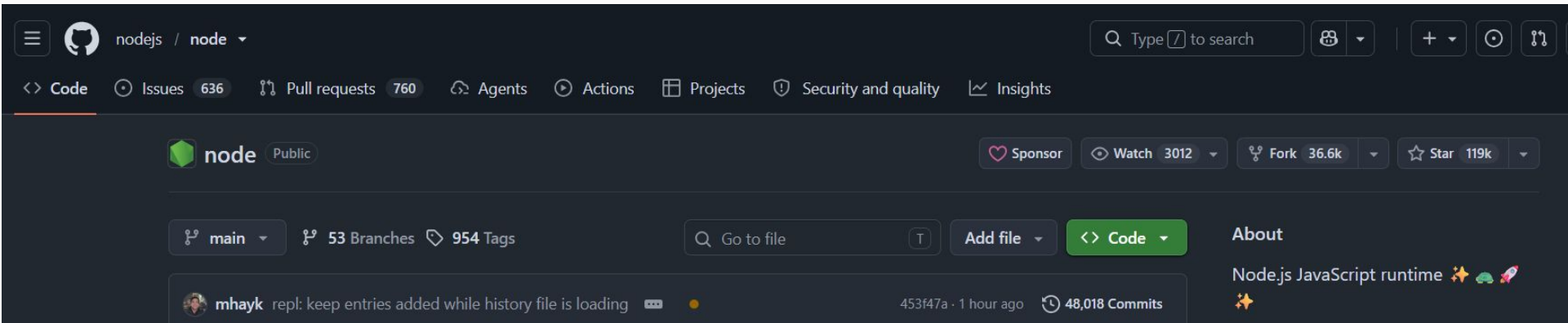
Para que cada um serve

Conhecendo o NodeJs

O que é

Antes: JavaScript rodava apenas no navegador

Agora: Existe um ambiente de execução que permite rodar JS fora do navegador (em outro computador, por exemplo) - Esse é o Node



The screenshot shows the GitHub interface for the Node.js repository. At the top, the navigation bar includes the Node.js logo, the repository name 'nodejs / node', a search bar, and icons for repository management. Below this, a secondary navigation bar lists various repository features: Code, Issues (636), Pull requests (760), Agents, Actions, Projects, Security and quality, and Insights. The main content area displays the repository details for 'node', marked as 'Public'. It includes statistics for Watchers (3012), Forks (36.6k), and Stars (119k), along with a 'Sponsor' button. Below the repository name, there's a section for branches and tags, showing 'main' as the selected branch, 53 branches, and 954 tags. A search bar for files is present. To the right, there's an 'About' section with the text 'Node.js JavaScript runtime' and a description of its capabilities. At the bottom, a snippet of a recent commit by 'mhayk' is visible, mentioning 'keep entries added while history file is loading'.

nodejs / node

Type to search

<> Code Issues 636 Pull requests 760 Agents Actions Projects Security and quality Insights

node Public

Sponsor Watch 3012 Fork 36.6k Star 119k

main 53 Branches 954 Tags

Go to file

Add file <> Code

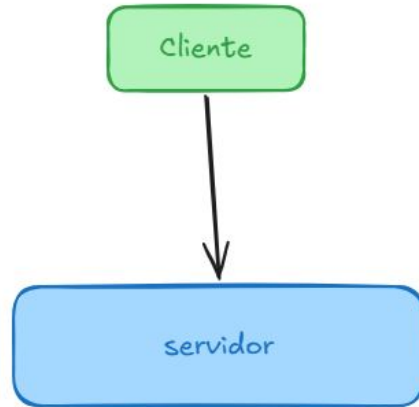
About

Node.js JavaScript runtime

mhayk repl: keep entries added while history file is loading 453f47a · 1 hour ago 48,018 Commits

Como funciona

Maioria dos servidores:



- trava tudo
- processa tudo que precisa
- libera

exemplo:



Como funciona

Node:

- não trava
- coleta várias requisições e vai mandando para o servidor
- quando uma termina, já retorna para o cliente

Setup inicial de um projeto Node e o NPM

npm init

O comando *npm init* é responsável por **registrar** o seu projeto

Argumento “-y”: **sim para tudo**

Ao invés de responder “sim” para cada pergunta que o init fizer, você apenas passa esse argumento, e ele automaticamente adota

Quais são essas perguntas?

npm init - Quais informações ele pede?

- nome do projeto (padrão: nome do diretório atual)
- versão do projeto (padrão: 1.0.0)
- descrição
- entry point (padrão: index.js)
- test command
- git repository
- keywords
- author
- licence (padrão: ISC)

Package.json

- “Certidão de nascimento” do projeto
- Guarda as informações de identidade dele, como o nome, versão, etc
- Guarda as **dependências** do projeto
- Guarda as instruções de execução de scripts, e forma de *importar coisas*

```
1  {
2    "name": "gestao-livros",
3    "version": "1.0.0",
4    "description": "",
5    "main": "server.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node server.js"
9    },
10   "keywords": [],
11   "author": "",
12   "type": "module",
13   "license": "ISC",
14   "dependencies": {
15     "cors": "^2.8.6",
16     "express": "^5.2.1"
17   }
18 }
```

A chave “Type” e como vamos usar

- formas coexistem
- package json define como o projeto vai se comportar

```
1 {  
2   "name": "gestao-livros",  
3   "version": "1.0.0",  
4   "description": "",  
5   "main": "server.js",  
6   "scripts": {  
7     "test": "echo \"Error: no test specified\" && exit 1",  
8     "start": "node server.js"  
9   },  
10  "keywords": [],  
11  "author": "",  
12  "type": "commonjs",  
13  "license": "commonjs",  
14  "dependencies": {  
15    "cors": "^2.8.6",  
16    "express": "^5.2.1"  
17  }  
18 }
```

Default value

Diferença entre CommonJs e Module

```
javascript
```

```
// CommonJS
```

```
const express = require('express')
```

```
// ES Module
```

```
import express from 'express'
```

NPM

Node Package Manager

“Loja de ferramentas gratuita e gigante”

- Preciso da ferramenta X para construir alguma coisa
 - Pede para o npm que ele te entrega

Sintaxe e uso - Instalação

```
PS C:\workspace\Aprender e crescer\aprender-e-crescer\BACKEND\aulas> npm install express
```

```
PS C:\workspace\Aprender e crescer\aprender-e-crescer\BACKEND\aulas> npm i express
```

O que acontece quando eu rodo esse comando?

- Pacote é baixado para a pasta ***node_modules***
- Package.json registra que seu projeto agora depende do pacote instalado

node_modules

public

index.html

routes

package-lock.json

package.json

server.js

package.json > ...

```
1  {
2    "name": "gestao-livros",
3    "version": "1.0.0",
4    "description": "",
5    "main": "server.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node server.js"
9    },
10   "keywords": [],
11   "author": "",
12   "type": "module",
13   "license": "ISC",
14   "dependencies": {
15     "cors": "^2.8.6",
16     "express": "^5.2.1"
17   }
18 }
```

Sintaxe e uso - Desinstalação

```
PS C:\workspace\Aprender e crescer\aprender-e-crescer\BACKEND\aulas> npm uninstall express
```

O que acontece quando eu rodo esse comando?

- Pacote é removido da pasta ***node_modules*** e do *package.json*

Mas por que eu preciso dessas dependências registradas no package.json?

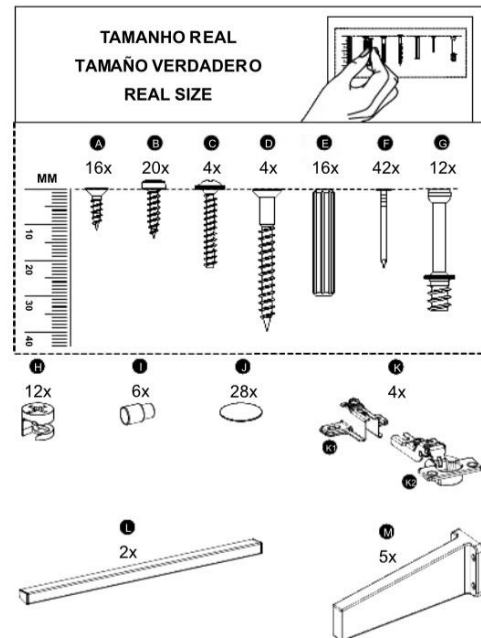
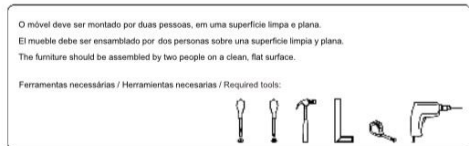
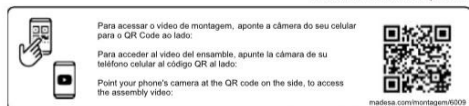
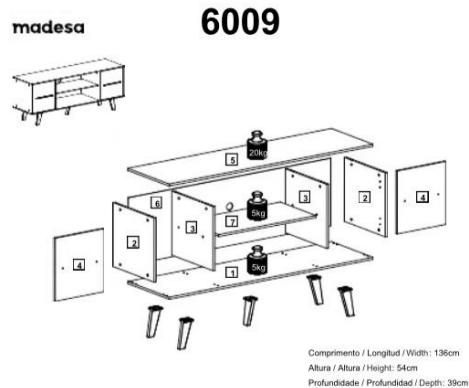
Imagine o seguinte cenário

Você construiu um software usando Node, e disponibilizou ele para que outras pessoas possam usar.

Para essa aplicação funcionar, você precisou instalar, na sua aplicação, um pacote chamado “express”.

O que vai acontecer quando um outro programador tentar usar a sua aplicação, mas não tiver o “express” instalado?

Imagine o seguinte cenário



Regra de ouro para desenvolvimento

Nunca versione a pasta *node_modules*

- Ela pode ser gigante, conforme a quantidade de dependências instaladas
- npm install será responsável por instalar tudo que precisar

Para saber mais

npm install nodemon --save-dev

Algumas ferramentas só são necessárias durante o desenvolvimento, não em produção. Sendo assim, quando você usa o argumento “--save-dev” durante a instalação, este pacote é adicionado em uma sessão separada do

package.json

```
json
{
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Conhecendo o Express

O que é

Framework web para Node

Biblioteca criada para entregar as ferramentas necessárias para se construir um servidor web

The screenshot shows the GitHub interface for the `expressjs/express` repository. At the top, there's a navigation bar with the repository name and a search bar. Below this is a secondary navigation bar with links to `Code`, `Issues` (104), `Pull requests` (120), `Agents`, `Discussions`, `Actions`, `Wiki`, `Security and quality` (3), and `Insights`. The main header area displays the repository name `express` as public, along with buttons for `Sponsor`, `Watch` (1714), `Fork` (24.7k), and `Star` (69.4k). Below the header, there's a section for branches and tags, showing `master` as the selected branch, with 16 branches and 304 tags. A search bar for files is present. To the right, there's an `About` section with the text: "Fast, unopinionated, minimalist web framework for node." At the bottom, a commit summary is visible, mentioning a build action by `dependabot[bot]` and showing 6,163 commits.

expressjs / express

Type / to search

<> Code Issues 104 Pull requests 120 Agents Discussions Actions Wiki Security and quality 3 Insights

express Public Sponsor Watch 1714 Fork 24.7k Star 69.4k

master 16 Branches 304 Tags Go to file Add file <> Code

dependabot[bot] build(deps): bump github/codeql-action/upload-sarif (#7400) 023767f · yesterday 6,163 Commits

About

Fast, unopinionated, minimalist web framework for node.

Sem o express

JS server.js > ...

```
1  const http = require('http')
2
3  const server = http.createServer((req, res) => {
4    if (req.url === '/usuarios' && req.method === 'GET') {
5      res.writeHead(200, { 'Content-Type': 'application/json' })
6      res.end(JSON.stringify({ nome: 'Lucas' }))
7    }
8  })
9
10 server.listen(3000)
```

Como funciona - 3 Pilares

- Aplicação
 - instância do express
- Rotas
 - verbo + path (caminho)
- Servidor
 - “escuta” determinada porta

O que o express NÃO faz

- Conexão ao banco de dados
- Autenticação
- Obriga uma determinada estrutura de pastas (livre)

Construindo uma API do zero com Node e Express



Ângelo Hank 6 min read · Jul 23, 2025



Let's Code

Vamos criar a nossa primeira API, um gerenciamento de alunos em memória